MCP–Cave Simulator: Hierarchical Trust, Oracle Invariance, and Meta-Reforms

This document formalizes the latest MCP–Cave developments into a precise mathematical model and an implementable algorithmic specification, culminating in a finalized **trust block** (with chunked exact counterfactual trust, invariance-aware oracle penalties, stratified sampling, smoothing, and adaptive schedules).

---

1. Conceptual Mapping

Plato's Cave (computationalized) - **Shadows**: agent-specific partial observations. - **Forms**: latent invariants generating observations. - **Dialectic**: iterative local refinement and social exchange. - **Ascent / Escape**: phase-transition-like convergence from fragmented/illusory states to stable identification of the latent invariant.

MCP encoding principle (operationalized) - We treat "prime states" multiplicatively but evolve them in **prime-exponent (log) space**, where updates are additive and stable.

---

2. State Space and Observation Model

2.1 Prime-exponent state representation

Let primes be indexed $p_1, \ldots, p_d$. Each agent i has an exponent vector

$$x^{(i)}(t) \in \mathbb{R}^d,$$

interpretable as a multiplicative encoding

$$P^{(i)}(t) \;=\; \prod_{k=1}^{d} p_k^{x_k^{(i)}(t)}.$$

All computational dynamics occur in x-space.

2.2 Shadows as partial projections of a latent Form

There exists a latent invariant (Form)

$$\theta^* \in \mathbb{R}^d$$

and each agent receives a partial/noisy "shadow" observation

$$y_i \;=\; \Pi_i(\theta^*) + \nu_i.$$

In the sandbox simulator, Π_i is implemented as a linear projection $A_i \in \mathbb{R}^{m \times d}$:

$$y_i \;=\; A_i \theta^* + \nu_i.$$

---

3. Local Dialectic Refinement

Agents locally refine their state by reducing their own shadow-error.

Define agent loss

$$E_i(x) \;=\; \frac{1}{m} \, \|A_i x - y_i\|_2^2.$$

A simple dialectic operator is gradient descent in exponent space:

$$\tilde{x}^{(i)}(t+1) \;=\; \mathrm{Shrink}_\lambda \Big(x^{(i)}(t) - \eta \, \nabla E_i(x^{(i)}(t)) \Big).$$

Here Shrink_λ is ℓ_1 soft-thresholding to enforce sparsity (interpretable as selecting "informative primes").

Adversarial agents can be modeled as perturbing their update direction.

---

4. Social Dialectic: Trust-Mixed Update

After local refinement, agents mix by a **row-stochastic trust matrix** $W(t)$:

$$x^{(i)}(t+1) \;=\; \sum_{j=1}^{N} W_{ij}(t) \, \tilde{x}^{(j)}(t+1) \;+\; (\text{oracle guidance}) \;+\; (\text{upward challenges}).$$

4.1 Exact counterfactual trust (Socratic "try-on")

The key normative idea:

> Agent i trusts j in proportion to how well j's worldview explains i's shadows.

Define counterfactual loss

$$L_{ij}(t) \;=\; E_i(\tilde{x}^{(j)}(t+1)).$$

A stable trust score uses row-wise normalization (subtract $\min_j L_{ij}$):

$$S_{ij}(t) \;\propto\; \exp\big(-\alpha_t \, [L_{ij}(t) - \min_j L_{ij}(t)] \big).$$

Then apply adjacency constraints and normalize:

$$W(t) \;=\; \mathrm{RowNormalize}(S(t) \odot A_{\mathrm{adj}}).$$

4.2 Dynamic trust sensitivity

To avoid premature lock-in or excessive volatility, the sensitivity parameter α_t decays over time with a floor:

$$\alpha_t \;=\; \max\{\alpha\, e^{-t/\tau},\; \alpha_{\min}\}.$$

4.3 EMA smoothing (epistemic inertia)

Trust is treated as a belief state with inertia:

$$\bar{S}(t) \;=\; (1-\rho)\,\bar{S}(t-1) + \rho\, S(t),$$

then $W(t) = \mathrm{RowNormalize}(\bar{S}(t))$.

---

5. Hierarchy as Soft, Revisable Belief

Each agent maintains a layer-belief distribution

$$h^{(i)}(t) \in \Delta^{L-1},$$

updated from performance evidence (e.g., exponential weighting). Hierarchy influences trust via a bias factor encouraging ascent without imposing fixed castes.

A simple bias uses expected level $\ell_i(t) = \langle h^{(i)}(t), u \rangle$ with $u = (0, 1, \ldots, L-1)$:

$$B_{ij}(t) \;=\; \exp\big(\gamma\,[\ell_j(t) - \ell_i(t)]\big).$$

Then trust score becomes $S_{ij}(t) \leftarrow S_{ij}(t)\, B_{ij}(t)$.

---

6. Forms as Invariance: Oracle Estimator

To preserve Platonic independence (Forms are not consensus), compute a global oracle estimate as an invariance fit:

$$\hat{\theta}(t) \;=\; \arg\min_{\theta} \sum_{i=1}^{N} \mathcal{L}(\Pi_i(\theta), y_i) + \Omega(\theta).$$

In the linear sandbox with ridge regularization:

$$\hat{\theta}(t) \;=\; \arg\min_{\theta} \sum_{i} \|A_i\theta - y_i\|^2 + \lambda\|\theta\|^2.$$

Oracle guidance (sunlight) can pull states toward $\hat{\theta}(t)$ via a small gain.

---

7. Invariance-Aware Trust Penalty

To prevent relativism and herding, penalize agents whose state worsens global invariance.

Define an oracle-consistency score for candidate j, estimated on a **stratified subsample** of agents:

$$E_j^{\text{oracle}}(t) \;\approx\; \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} E_i(\tilde{x}^{(j)}(t+1)).$$

Then penalize increases relative to a baseline (previous step or median after a dimension change):

$$\Delta_j(t) = E_j^{\text{oracle}}(t) - E_j^{\text{oracle}}(t-1), \qquad \pi_j(t) = \exp\Big(-\kappa\,\frac{\max(\Delta_j(t),0)}{\tau_\pi}\Big),$$

clipped by $\pi_j \geq \pi_{\min} > 0$.

This multiplicatively filters trust: $S_{ij} \leftarrow S_{ij}\,\pi_j$.

---

8. Meta-Reforms and Prime-Space Adaptation

Meta-reforms can be triggered when diagnostic variance (e.g., residual dispersion) exceeds a threshold.

Reforms may include: - small stochastic perturbations of $h^{(i)}$ (soft reset, not full reshuffle) - dimensional pruning: drop low-utility or low-variance exponent coordinates - invariance-preserving rollback: accept pruning only if oracle fit does not worsen beyond a tolerance

---

9. Observable Predictions

1. **Cascading escapes**: detectable as sudden drops in global error and/or spikes in change-point detectors.
2. **False suns**: stable clusters with high agreement but poor oracle fit and persistent cross-layer disagreement.
3. **Robustness under adversaries**: exact counterfactual trust + invariance penalty + reforms reduces herding and improves final accuracy.

---

10. Fastest Validation Protocol (Sandbox)

Setup - $N \in [60, 400]$, $d \approx 8$, $m \approx 6$, $L \in [3, 5]$ - adversaries: 10–30% of agents - noise sweep: $\sigma \in [0.05, 0.15]$

Compare variants 1) flat adaptive 2) hierarchical soft 3) hierarchical + reforms + invariance penalty (full) 4) ablation (no upward flow / no reforms)

Metrics - final error $\frac{1}{N}\sum_i \|x^{(i)} - \theta^*\|$ - oracle loss - hierarchy entropy - cascade events (CUSUM/change-point)

Finalized Code Snippet

This snippet includes: - exponential α_t schedule with floor - trust-mode toggle with chunked exact evaluation - stratified oracle-penalty computed in chunks (memory-safe) - penalty cap and active-dimension reset

> **Integration points** 1) Add the parameters and state variables listed below. 2) Replace your existing α_t computation. 3) Replace trust block sections (A) and (C) with the code below (or directly paste into your trust block).

A) Add parameters to `run_trial(...)`

```
trust_mode="exact",          # "proxy" | "exact" | "exact_chunked"
chunk_threshold=150,
chunk_size=50,

# Alpha schedule
dynamic_alpha=True,
alpha_schedule="exp",        # "exp" or "linear"
alpha_tau=20.0,
alpha_min=0.5,

# EMA smoothing
trust_ema=0.30,

# Oracle penalty
oracle_penalty=0.20,
oracle_subsample=0.20,
oracle_penalty_tau=0.05,
penalty_min=0.20,

# Chunked oracle penalty controls
oracle_chunk_size=None,      # defaults to chunk_size
memory_threshold_floats=1_000_000,
shuffle_oracle_batches=True,

# Stratified subsample controls
stratify_balance=True,
```

```
min_k_per_level=2,
high_layer_bias=1.0,
```

B) Add state variables near trust init

```
E_oracle_agent_prev = None
active_prev = active.copy()
score_ema = None
```

C) Exponential alpha schedule (replace your alpha_t computation)

```
if dynamic_alpha:
    if alpha_schedule == "exp":
        alpha_t = alpha * np.exp(-t / (alpha_tau + 1e-12))
    else:
        alpha_t = alpha / (1.0 + t / 10.0)
else:
    alpha_t = alpha
alpha_t = max(alpha_t, alpha_min)
```

D) Trust base score (A) with `trust_mode` and chunking

```
# Ensure pruned dims don't affect trust computations (recommended if pruning/
reforms exist)
x_eval = x_tilde.copy()
x_eval[:, ~active] = 0.0

if trust_mode == "proxy":
    dist = np.linalg.norm(x_eval[:, None, :] - x_eval[None, :, :], axis=2)  #
(N,N)
    base_score = np.exp(-0.5 * dist) * np.exp(-alpha_t * (E_after[None, :]))

else:
    # exact counterfactual losses_all[i,j] = E_i(x_eval[j])
    if trust_mode == "exact_chunked" and N > chunk_threshold:
        losses_all = np.empty((N, N), dtype=np.float32)
        for j0 in range(0, N, chunk_size):
            j1 = min(j0 + chunk_size, N)
            x_ch = x_eval[j0:j1]  # (chunk,d)
            pred_ch = np.einsum("imd,jd->ijm", A, x_ch)          # (N,chunk,m)
            resid_ch = pred_ch - y[:, None, :]                   # (N,chunk,m)
            losses_all[:, j0:j1] = np.mean(resid_ch**2, axis=2)   # (N,chunk)
    else:
```

```
        pred_all = np.einsum("imd,jd->ijm", A, x_eval)          # (N,N,m)
        resid_all = pred_all - y[:, None, :]                    # (N,N,m)
        losses_all = np.mean(resid_all**2, axis=2)              # (N,N)

    row_min = np.min(losses_all, axis=1, keepdims=True)
    base_score = np.exp(-alpha_t * (losses_all - row_min))

score = base_score * A_adj
```

E) (C) Chunked invariance penalty with stratified subsample (drop-in replacement for penalty stage)

```
if oracle_penalty > 0.0:
    oc = oracle_chunk_size if oracle_chunk_size is not None else chunk_size

    # Stratified subsample by hierarchy level
    levels = np.argmax(h, axis=1)
    counts = np.array([(levels == lv).sum() for lv in range(L)], dtype=float)

    k_total = max(L * min_k_per_level, int(np.ceil(oracle_subsample * N)))

    if stratify_balance:
        w = counts.copy()
    else:
        w = np.ones(L, dtype=float)

    # Bias toward higher layers (ascent fidelity)
    w *= (np.arange(L) + 1.0) ** high_layer_bias
    w = w / (w.sum() + 1e-12)

    k_lv = np.maximum(min_k_per_level, np.round(k_total * w).astype(int))
    k_lv = np.minimum(k_lv, counts.astype(int))

    cur = int(k_lv.sum())
    if cur < k_total:
        order = np.argsort(-w)
        for lv in order:
            if cur >= k_total:
                break
            avail = int(counts[lv] - k_lv[lv])
            if avail <= 0:
                continue
            add = min(avail, k_total - cur)
            k_lv[lv] += add
            cur += add
```

```
    elif cur > k_total:
        order = np.argsort(w)
        for lv in order:
            if cur <= k_total:
                break
            removable = int(k_lv[lv] - min_k_per_level)
            if removable <= 0:
                continue
            rem = min(removable, cur - k_total)
            k_lv[lv] -= rem
            cur -= rem

    idx_sub = []
    for lv in range(L):
        cand = np.where(levels == lv)[0]
        if cand.size == 0 or k_lv[lv] <= 0:
            continue
        pick = rng.choice(cand, size=min(int(k_lv[lv]), cand.size),
replace=False)
        idx_sub.extend(pick.tolist())

    idx_sub = np.unique(idx_sub)
    A_sub = A[idx_sub]
    y_sub = y[idx_sub]
    k = len(idx_sub)

    use_chunk = (trust_mode == "exact_chunked" and N > chunk_threshold) or (N *
k * m > memory_threshold_floats)

    x_or = x_eval  # already zeroed inactive dims

    if use_chunk:
        if shuffle_oracle_batches:
            order = rng.permutation(N)
            inv = np.empty(N, dtype=int)
            inv[order] = np.arange(N)
            x_proc = x_or[order]
        else:
            order = None
            inv = None
            x_proc = x_or

        E_oracle_agent_proc = np.empty(N, dtype=float)
        for j0 in range(0, N, oc):
            j1 = min(j0 + oc, N)
            x_ch = x_proc[j0:j1]
            pred_ch = np.einsum("kmd,jd->jkm", A_sub, x_ch)            #
(chunk,k,m)
```

```
            resid_ch = pred_ch - y_sub[None, :, :]
            E_oracle_agent_proc[j0:j1] = np.mean(resid_ch**2, axis=(1, 2))

        E_oracle_agent = E_oracle_agent_proc[inv] if inv is not None else
E_oracle_agent_proc
    else:
        pred_sub = np.einsum("kmd,jd->jkm", A_sub, x_or)              # (N,k,m)
        resid_sub = pred_sub - y_sub[None, :, :]
        E_oracle_agent = np.mean(resid_sub**2, axis=(1, 2))

    # Reset baseline if first time OR active dims changed (e.g., pruning/
reforms)
    if (E_oracle_agent_prev is None) or (not np.array_equal(active,
active_prev)):
        baseline = np.median(E_oracle_agent)
        delta = E_oracle_agent - baseline
    else:
        delta = E_oracle_agent - E_oracle_agent_prev

    penalty = np.exp(-oracle_penalty * np.maximum(delta, 0.0) /
(oracle_penalty_tau + 1e-12))
    penalty = np.maximum(penalty, penalty_min)
    score *= penalty[None, :]

    E_oracle_agent_prev = E_oracle_agent
    active_prev = active.copy()
```

F) EMA smoothing + normalize to trust matrix

```
if trust_ema > 0.0:
    if score_ema is None:
        score_ema = score
    else:
        score_ema = (1.0 - trust_ema) * score_ema + trust_ema * score
    score_used = score_ema
else:
    score_used = score

W = row_stochastic(score_used)
```

Notes on Usage

- For **N ≤ 150**, prefer `trust_mode="exact"`.

- For **N > 150**, prefer `trust_mode="exact_chunked"`.
- If oracle penalty seems noisy, increase `oracle_penalty_tau` from `0.05` to `0.1`.
- If early convergence is too fast or unstable, increase `alpha_tau` and/or `trust_ema`.

Optional Next Extensions

1. Chunked computation for `losses_all` and `E_oracle_agent` can be made fully streaming to support **N > 2000**.
2. Replace linear A_i projections with nonlinear feature maps for real datasets.
3. Add a diagnostic for "false suns" by clustering final $x^{(i)}$ and comparing within-cluster agreement vs oracle fit.